

Modul

Instant Credit Approval & Disbursement (BNPL)

Bagian 1: Software Requirement Specification (SRS)

1.1 High-Level Architecture (Microservices, High Concurrency)

Dalam desain ini, platform BNPL dipisah menjadi microservices dengan API Gateway sebagai titik masuk tunggal (rate limiting, auth, routing). Untuk skenario Harbolnas (traffic tinggi), komponen utama:

1. stateless service + horizontal scaling,
2. database per service untuk isolasi beban
3. cache Redis untuk idempotency & hot read
4. Kafka untuk beban asinkron agar tidak memblokir jalur transaksi real-time.

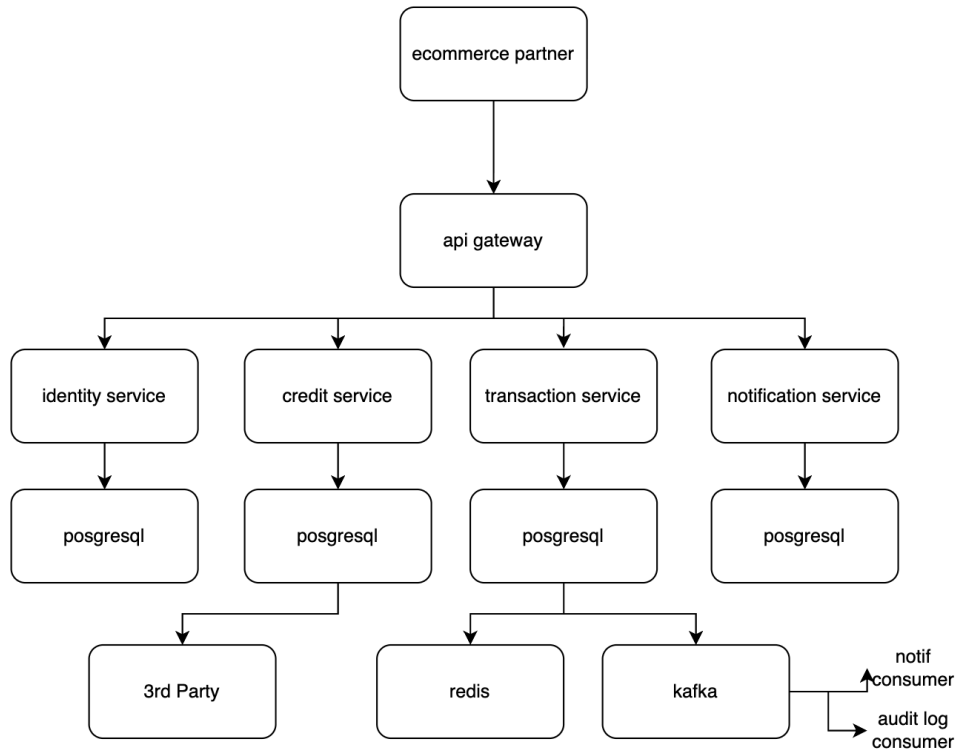
Komponen layanan (contoh):

- Identity Service — registrasi, KYC ringkas, sesi pengguna, penerbitan token jwt.
- Credit Engine Service — scoring, integrasi credit bureau, kebijakan approve/reject limit.
- Transaction Service — pencatatan transaksi, debit saldo limit, ledger, idempotensi (service yang diimplementasikan di repositori pendamping).
- Notification Service — konsumsi event; kirim email/SMS/push; tidak memblokir transaksi.
- Audit/Analytics (opsional) — konsumsi topik audit untuk BI dan compliance.

Message broker (Kafka):

- Topik contoh: credit.application.submitted, credit.approved / credit.rejected, transaction.created, notification.request, audit.log.
- Proses asinkron: notifikasi ke nasabah, fan-out ke sistem audit, materialisasi read model.
- Partition key: userId atau accountId agar urutan per nasabah tetap konsisten.

Diagram blok



Strategi database (RDBMS vs NoSQL/Cache)

Kebutuhan	Teknologi	Alasan
Saldo limit, transaksi, ledger, outbox (system of record)	PostgreSQL (RDBMS)	Butuh ACID; banyak penulisan multi-baris dalam satu transaksi; constraint & locking (SELECT FOR UPDATE).
Idempotency key lookup, rate limit counter	Redis	Akses O(1), TTL native, throughput tinggi untuk hot path.
Event log / fan-out	Kafka	Desacoupling, replay, backpressure terkelola.
Profil pengguna read-heavy (opsional)	PostgreSQL + Redis cache	Master di RDBMS, cache untuk latency.

1.2 REST API — Pengajuan / Pencatatan Transaksi Kredit (BNPL)

Catatan skenario: pengajuan kredit awal bisa melalui Credit Engine; di implementasi referensi ini difokuskan endpoint pencatatan transaksi setelah limit disetujui: POST /api/v1/transactions.

Kontrak lengkap tersedia juga di berkas openapi.yaml pada repositori.

Endpoint utama

POST /api/v1/transactions

Header wajib:

- Idempotency-Key: string unik per permintaan logis (rekomendasi UUID).
- Content-Type: application/json
- X-Correlation-Id: opsional; untuk tracing (dibalas di response).

Request body (contoh field):

Field	Tipe	Validasi
userId	UUID	Wajib
externalRef	string	3–64, alfanumerik + -/_
amount	decimal	1000–50000000 (batas sesuai kebijakan)
currency	string	IDR USD SGD
tenorMonths	integer	1–36
transactionType	enum	PURCHASE REFUND REPAYMENT (implementasi saat ini: PURCHASE)
merchantId	string	opsional, max 32

Response sukses (201 Created):

- Body JSON: transactionId, status, remainingLimit, currency, createdAt, idempotencyReplayed.
- Header: Location (URI resource), X-Idempotency-Replayed (true jika replay dari cache).

Penanganan error (RFC 7807 Problem Details, Content-Type application/problem+json):

HTTP	Situasi
400	Validasi input, header hilang, argument tidak valid
404	Akun kredit tidak ditemukan
409	Konflik idempotency (payload berbeda), duplicate externalRef, request masih diproses
422	Limit tidak cukup, akun non-aktif (business rule)
500	Kesalahan tidak terduga

1.3 ERD & Data Dictionary (Transaction Service)

Relasi ringkas: credit_accounts 1—N transactions; transactions 1—N ledger_entries;
outbox_events untuk pola Outbox.

Tabel: credit_accounts

Kolom	Tipe	Keterangan
id	UUID PK	Identitas akun
user_id	UUID UNIQUE	Referensi pengguna
account_number	VARCHAR(20) UNIQUE	Nomor akun tampilan
credit_limit	NUMERIC(19,4)	Plafon
available_limit	NUMERIC(19,4)	Sisa limit
currency	CHAR(3)	ISO 4217
status	VARCHAR	ACTIVE SUSPENDED CLOSED
version	BIGINT	Optimistic locking pendamping
created_at, updated_at	TIMESTAMPTZ	Audit waktu

Tabel: transactions

Kolom	Tipe	Keterangan
id	UUID PK	ID transaksi
account_id	UUID FK	Ke credit_accounts
external_ref	VARCHAR UNIQUE	Referensi merchant
idempotency_key	VARCHAR UNIQUE	Kunci idempotensi
amount	NUMERIC(19,4)	Nominal
currency	CHAR(3)	
tenor_months	INT	Cicilan
merchant_id	VARCHAR	Opsional
status	VARCHAR	PENDING APPROVED REJECTED REVERSED
created_at, updated_at	TIMESTAMPTZ	

Tabel: ledger_entries

Kolom	Tipe	Keterangan
id	BIGSERIAL PK	Urutan kronologis
transaction_id	UUID FK	
account_id	UUID FK	
entry_type	VARCHAR	DEBIT CREDIT
amount	NUMERIC(19,4)	Selalu positif
balance_after	NUMERIC(19,4)	Saldo setelah entri
currency	CHAR(3)	
created_at	TIMESTAMPTZ	Append-only

Tabel: outbox_events

Kolom	Tipe	Keterangan
-------	------	------------

id	BIGSERIAL PK	
aggregate_id	UUID	ID transaksi / agregat
aggregate_type	VARCHAR	Contoh: Transaction
event_type	VARCHAR	transaction.created
payload	TEXT	JSON event
status	VARCHAR	PENDING PUBLISHED FAILED
retry_count	SMALLINT	
created_at, published_at	TIMESTAMPTZ	

Bagian 2: Pengembangan & Kualitas Kode (Technical Implementation)

Service yang dipilih: Transaction Service. Bahasa: Java 17 + Spring Boot 3. Kode referensi tersedia pada folder repositori yang disertakan (README.md, docker-compose.yml, Flyway, unit test).

2.1 Clean / Hexagonal Architecture

- domain/ — entitas, value object (Money), exception, port in/out; tanpa dependency framework.
- application/ — CreateTransactionService orkestrasi idempotensi; DefaultCreateTransactionWorkflow transaksi DB.
- infrastructure/ — adapter JPA, Redis, relay Kafka outbox, serializer.
- interfaces/ — REST controller, DTO, GlobalExceptionHandler (RFC 7807), CorrelationIdFilter.

2.2 Keamanan & Integritas — Idempotency & Data Sensitif

- Idempotency-Key: hash payload bisnis + Redis SETNX dengan TTL; complete menyimpan response; conflict jika key sama dan hash beda — mencegah double debit pada retry.
- NIK/limit: tidak dicetak ke log; gunakan BigDecimal untuk nominal; batasi detail error di production; HTTPS di gateway.
- Outbox Pattern: insert event dalam transaksi yang sama dengan penulisan transaksi & ledger — menghindari dual-write inkonsisten ke Kafka.

2.3 Database & ACID

- Satu @Transactional pada workflow: SELECT credit_accounts FOR UPDATE, debit di domain, INSERT transaction, UPDATE account, INSERT ledger, INSERT outbox — commit atomik atau rollback penuh.
- Pessimistic lock mengurangi race condition concurrent pada saldo yang sama.
- Skema: migrasi Flyway V1–V4 (+ seed V5 untuk demo lokal).

2.4 Unit Testing (minimal 2–3)

- CreditAccountTest: debit mengurangi saldo; insufficient limit; akun tidak aktif.
- DefaultCreateTransactionWorkflowTest: happy path menyimpan transaksi + outbox; insufficient limit tidak menyimpan transaksi.
- CreateTransactionServiceTest: replay idempotensi tanpa memanggil workflow ulang.

Bagian 3: Leadership & Code Review Strategy

Sebagai Lead untuk 5 developer dengan skill yang beragam, strategi Code Review (CR) yang saya terapkan berfokus pada kolaborasi, edukasi, dan penjagaan standar tanpa membuat bottleneck.

1. Strategi Code Review

- **Checklist Standar (Automated + Manual):** Sebelum kode masuk ke tahap manual review oleh saya atau peer reviewer, CI/CD pipeline harus **pass** (Unit Test, Linter, SonarQube untuk deteksi security vulnerabilities & code smells). Ini menghemat waktu reviewer dari mengomentari **formatting** atau isu sepele.

- **Peer Review Berpasangan:** Saya akan memasangkan Junior/Mid-level dengan Senior developer. Ini menumbuhkan budaya **knowledge sharing**. Saya sebagai Lead akan me-review arsitektur kritikal (seperti modul transaksi/database locking) untuk memastikan tidak ada **race condition** atau **SQL injection**.

- **Checkpoint Utama saat Manual Review:**

- **Security:** Memastikan tidak ada **hardcoded credentials**, validasi input/sanitasi berjalan dengan benar (mencegah XSS/SQLi), dan implementasi **Idempotency** tidak bocor (race-condition saat mengecek idempotency key).

- **Performance:** Mengecek apakah ada "N+1 Query" pada ORM/Database access, penggunaan index pada kolom yang sering di-filter, dan penggunaan asinkron/message broker untuk operasi non-kritikal/berat.

- **Maintainability & Clean Code:** Memeriksa penamaan variabel yang deskriptif, penerapan **Separation of Concerns** (controller tidak boleh berisi logic bisnis kompleks, harus diserahkan ke **Use Case/Service layer**), dan ketersediaan komentar pada logika yang rumit.

2. Memastikan Kualitas di Tengah Tekanan Deadline

Tekanan rilis cepat sangat umum di industri finansial, namun kompromi pada kualitas (terutama di sistem transaksi) bisa berakibat fatal.

- **MVP Mindset & Scope Management:** Jika deadline ketat, saya bernegosiasi dengan Tim Bisnis/Product untuk merilis fitur MVP (Minimum Viable Product). Kita kurangi **scope** kosmetik atau **nice-to-have features**, namun fitur intinya (security, ledger) ***tidak boleh dikompromikan***.

- **Test-Driven Development (TDD) / Shift-Left Testing:** Saya mengharuskan tim menulis Unit Test minimal untuk **Happy Path** dan **Critical Edge Cases** (seperti limit tidak cukup atau concurrent

request) sebelum/bersamaan dengan penulisan fitur. Bug yang ditemukan di awal lebih cepat diperbaiki daripada yang ditemukan saat UAT.

- **Dokumentasi yang Terintegrasi (Living Documentation):** Membuat dokumentasi tidak perlu memakan waktu lama jika terotomatisasi. Menggunakan Swagger/OpenAPI (lewat dekorator kode) untuk dokumentasi API, dan menggunakan ADR (Architecture Decision Records) sederhana untuk mencatat "Mengapa kita memilih pendekatan X".

- **Tech Debt Backlog:** Jika ada jalan pintas (shortcut) yang terpaksa diambil demi deadline (asalkan BUKAN security/ACID compliance), kita akan mencatatnya dengan sadar sebagai **Technical Debt** di Jira/Sprint Backlog untuk diselesaikan di sprint **maintenance** berikutnya.